



Stage Master 2 SIRIS, Rapport de stage

Florent HARDY

Préparation au Profilage et à l'Optimisation de la Consommation Énergétique des Réseaux Programmables

Stage dans le laboratoire iCube, au sein de l'équipe Réseaux

du 10 Février 2025 au 26 Août 2025

Tuteurs de stage

Jean-Romain LUTTRINGER, Julien MONTAVONT
Maîtres de conférences
Laboratoire iCube

Enseignant référent

Pierre DAVID
Maître de conférences
Université de Strasbourg

Table des matières

1	Introduction	4
2	Environnement de travail	5
2.1	Présentation de l'équipe	5
2.2	Conditions de travail	5
3	Contexte : Programmabilité des réseaux	6
3.1	Introduction	6
3.2	Programmabilité des réseaux	7
3.3	La P4-programmabilité	7
3.3.1	Architecture	8
3.3.2	Fonctionnement	8
4	L'étude de la consommation des réseaux : un état de l'art	10
4.1	Optimisation	10
4.2	Prédiction	12
4.3	Taxonomie	12
5	Préparation à la modélisation	13
5.1	La plateforme <i>SlicesFR</i>	13
5.2	Observations préliminaires	14
5.3	Méthodologie	15
6	Préparation à l'optimisation	16
6.1	Dépliage de préfixe	16
6.2	Modifier l'ordonnancement des tables	17
6.3	Cache énergétique	17
6.3.1	Proposition d'implémentation déterministe	19
6.3.2	Proposition d'implémentation probabiliste	19
6.3.3	Analyse numérique de l'impact d'un cache probabiliste	20
6.3.4	Précisions concernant les limites et perspectives des implémentations	22
7	Conclusion	23
7.1	Perspectives	23
7.1.1	Perspectives à court terme	23
7.1.2	Perspectives à long terme	24
	Bibliographie	28

Remerciements

Je tiens à remercier Jean-Romain LUTTRINGER de m'avoir fait confiance pour ce projet et pour ses idées, son soutien, et sa pédagogie.

Mes remerciements les plus sincères vont à mes encadrants, Jean-Romain LUTTRINGER et Julien MONTAVONT, pour leurs nombreux retours. Les qualités de ce document et de mon travail de manière générale sont toutes des conséquences de leurs conseils.

Toute ma reconnaissance va à Pierre DAVID, enseignant référent, dont les conseils ont été précieux tout le long du stage.

Merci également à l'équipe de recherche Réseaux qui a su faire du lieu de travail un endroit accueillant et chaleureux.

1 Introduction

Ce stage entre dans le cadre de mon Master 2 Sciences et Ingénierie des Réseaux, de l'Internet et des Systèmes. Il a été réalisé au sein de l'équipe de recherche Réseaux du laboratoire iCube, dans le contexte du projet GARDEN, visant à réduire la consommation énergétique des réseaux, porté par Jean-Romain LUTTRINGER.

En 2023, les technologies de l'information et de la communication représentaient 10% de la consommation globale en électricité [Gel23]. Un dixième de cette consommation provient des équipements réseau et des centres de données [IEA].

Les équipements programmables sont des outils facilitant le prototypage de solutions via leur flexibilité. Étant donné que toute la chaîne de traitement des paquets, de la recherche de chemins à l'écriture de labels dans les en-têtes, peut être redéfinie, les équipements programmables sont devenus un vecteur d'innovations pour les chercheur-e-s. L'introduction d'équipements programmables a alors facilité le prototypage de solutions de réduction de la consommation.

L'étude de la consommation des réseaux n'est pas nouvelle, mais l'introduction des équipements réseau programmables a ravivé cette thématique. Parmi ces solutions se trouvent des outils visant à réduire la consommation des réseaux. Ils prennent la forme d'optimisations compilateurs, de protocoles, de technologies, d'algorithmes, *etc.* Ils amènent également de nouvelles pistes de recherche, en particulier pour définir un traitement sur mesure et dynamique des routes [BGK⁺13], de la sécurité [SPL⁺16] et d'autres.

Dans ce rapport nous étudierons les questions suivantes : "*Quels nouveaux critères doivent être pris en considération lors de l'étude de la consommation énergétique des réseaux programmables ?*", et "*Quels facteurs sont intéressants à considérer pour réduire la consommation de ces réseaux, et comment les optimiser ?*". Je me suis aidé des logiciels Intel pour préparer l'estimation de la consommation et j'ai préparé plusieurs solutions, notamment une technologie de cache qui vise à optimiser la consommation énergétique.

La première partie porte sur l'environnement de travail et le contexte dans lequel ce stage s'inscrit 2. Le document introduit les notions importantes à la compréhension notamment l'architecture des réseaux programmable dans la partie 3. Cette partie est suivie d'un état de l'art recensant les différentes études faites à propos de la consommation énergétique des réseaux 4. Suite à quoi, je présente, dans la partie 5, l'étude qui a été réalisée dans le cadre de ce stage pour estimer la consommation énergétique d'un équipement réseau programmable. La partie 6 s'intéressera aux différentes méthodes possibles, pour réduire la consommation à l'échelle du programme inséré dans les équipements. Finalement, la partie 7 de ce rapport résume le travail accompli et énumère les perspectives.

2 Environnement de travail

Le laboratoire iCube a été créé en 2013 sous l'égide du CNRS, de l'Université de Strasbourg, de l'ENGES et de l'INSA. Il compte 750 membres dont font notamment partie des maître·sse·s de conférence, des professeur·e·s des universités et des ingénieur·e·s de recherche. Mes tuteurs de stage, Jean-Romain LUTTRINGER et Julien MONTAVONT, sont enseignants-chercheurs pour l'équipe *Réseaux* du Département Informatique Recherche.

Le laboratoire dans lequel se situe l'équipe *Réseaux* est sur le campus d'Illkirch-Graffenstaden, au sein du bâtiment du Pôle API, dont la construction s'est achevée en 1993. Il héberge deux écoles d'ingénieur·e·s, plusieurs laboratoires et des plateformes de transfert de technologies.

2.1 Présentation de l'équipe

L'équipe de recherche Réseaux d'iCube est composée de 11 membres permanents, 3 post-doctorant·e·s et 6 doctorant·e·s. Comme le nom l'indique, l'équipe focalise ses recherches sur la conception d'algorithmes et protocoles de routage et de communication. Les principaux sujets sont :

- les réseaux de bordure : l'étude des connexions entre objets intelligents et leur environnement en termes de fiabilité ou de consommation d'énergie. Julien MONTAVONT, Thomas NOËL, Samir SI-MOHAMMED et Fabrice THEOLEYRE, par exemple, travaillent sur l'Internet des Objets ;
- les réseaux de cœur : l'étude de l'acheminement des paquets dans les réseaux informatiques filaires permettant les communications entre machines. Quentin BRAMAS, Jean-Romain LUTTRINGER et Pascal MÉRINDOL travaillent notamment sur les outils d'ingénierie de trafic utilisés dans les réseaux d'opérateurs ;
- les algorithmes distribués : l'étude des systèmes distribués, leur complexité, contraintes et applications. Quentin BRAMAS et Anissa LAMANI, en particulier, travaillent sur des problèmes portant sur des essaims d'agents autonomes.

2.2 Conditions de travail

Le projet GARDEN, dans lequel ce stage s'inscrit, est un projet Jeunes Chercheuses Jeunes Chercheurs porté par Jean-Romain LUTTRINGER et est financé par l'Agence Nationale de la Recherche. Il cherche à réduire la consommation des réseaux informatiques et alimenter la base de savoirs sur la consommation énergétique des technologies de l'information et de la communication.

Je travaille dans un bureau partagé à proximité de l'équipe. On m'a confié un ordinateur ainsi que des accès à la documentation Intel et à du matériel et logiciels nécessaires notamment accessibles via la plateforme *SlicesFR*, qui met à disposition une plateforme expérimentale pour tester ou évaluer des prototypes.

Mes encadrants et moi faisons une réunion tous les mois environ pour nous mettre à jour sur nos avancées et pour décider de la direction du projet. Des échanges à l'oral ou par messagerie venaient compléter ces réunions, et servaient surtout à partager des sources d'informations jugées intéressantes.

3 Contexte : Programmabilité des réseaux

Afin de fonctionner correctement, les réseaux informatiques doivent assurer un nombre important de fonctionnalités (dissémination de l'information, calcul de routes, transmission des données...). Pour faciliter l'étude de tels systèmes, ces fonctions ont été regroupées dans différents blocs logiques appelés "*plans*".

3.1 Introduction

Le *plan de management* sert à faire circuler les messages relatifs à la supervision et à la configuration des équipements.

Le *plan de contrôle* dissémine l'information requise et calcule les chemins de transmission pour construire une carte du réseau (voir Figure 1). Il exécute les protocoles de recherche de plus courts chemins, tel que OSPF [Moy98] ou IS-IS [Cal90] (voir 1. de la Figure 1). Un résumé de ce calcul, liant la destination au port de sortie devant être utilisé pour cette dernière, est installé dans le matériel pour être utilisé par le plan de données (voir 2. de la Figure 1).

Le *plan de données*, quant à lui, sert à traiter les paquets de la manière la plus efficace possible. Il suit les instructions envoyées par le plan de contrôle pour acheminer les paquets du port d'entrée au port de sortie pertinent (voir 3. de la Figure 1). Afin d'être efficaces, les fonctions du plan de données sont implémentées dans le matériel via des *Application Specific Integrated Circuit* (ASIC). Le plan de contrôle utilise, quant à lui, des logiciels pour fonctionner.

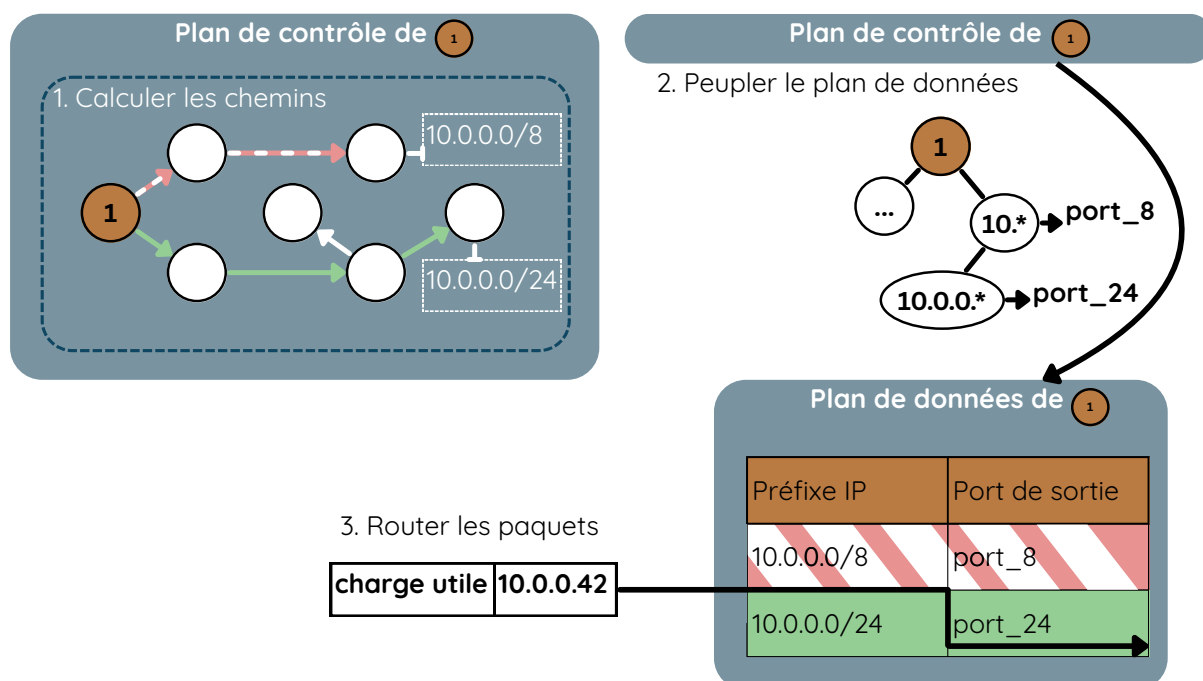


FIGURE 1 – Illustration du rôle du plan de contrôle.

Le comportement de ces plans doit parfois être modifiable suivant les besoins du réseau. De ce fait, les

plans sont généralement configurables, voire programmables en fonction des équipements présents dans le réseau.

La différence entre programmer et configurer un équipement réseau se trouve dans la capacité à transcender les limites fixées par les constructeurs. Si les constructeurs ont défini une liste de protocoles que l'on peut choisir et dont on peut modifier les paramètres alors on considère que l'équipement est configurable. Si les administrateurs-réseaux sont en mesure de redéfinir les protocoles comme bon leur semble alors on considère que l'équipement est programmable. La programmabilité permet d'innover rapidement, sans dépendre de la validation des constructeurs.

3.2 Programmabilité des réseaux

La programmabilité des réseaux a évolué au fil du temps, certains plans étant plus compliqués à rendre programmables que d'autres.

Le plan de management est programmable depuis les années 80 [HHM⁺23]. En effet, la circulation et le traitement d'informations relatives à une infrastructure n'a rien de nouveau pour l'époque, il suffisait de normaliser les communications. Des protocoles comme SNMP¹ permettent de programmer des outils de supervision personnalisés.

Le plan de contrôle, quant à lui, est devenu programmable grâce à l'essor des *Software Defined Networks* et en particulier d'*OpenFlow* [MAB⁺08], qui est une standardisation des communications entre le plan de contrôle programmable et le plan de données. Suite à cette standardisation, étant donné que le plan de contrôle est entièrement logiciel, les administrateurs-réseaux peuvent redéfinir le plan de contrôle comme bon leur semble. Les algorithmes de routage peuvent être définis sur mesure, *e.g.* ils peuvent devenir dynamique et sensible aux congestions voire rediriger le trafic en fonction de son contenu.

Le plan de données est devenu programmable (tout en ayant une haute performance) dans les années 2013/2014 à l'aide des *Reconfigurable Match Tables* [BGK⁺13] et du langage P4 [BDG⁺14]. Ce passage à un plan de données programmable a été le plus compliqué car, pour des raisons d'efficacité, ses fonctions sont implémentées dans le matériel à l'aide d'*ASICs* qui sont spécifiques à certaines applications. Il n'est donc pas facile de les convertir en un équivalent programmable. P4 est l'un des langages haut niveau (avec eBPF [eF], Verilog [AMD], *etc.*) visant à définir le comportement du plan de données.

3.3 La P4-programmabilité

Les équipements P4-programmables viennent principalement sous forme de commutateur (*ASICs* comme le Tofino²[Intb]), de *Field-Programmable Gate Array*³ [Inta] et de *Network Interface Card*⁴ [XQH⁺23]. P4 étant un langage de haut niveau, ce sont les constructeurs de ces équipements qui doivent définir le compilateur qui va allouer les ressources nécessaires pour faire fonctionner le code. Bien qu'il existe un seul langage P4, il peut y avoir autant de compilateurs P4 que d'équipements compatibles.

La première version de P4 : P4₁₄ a été introduite en 2014. Son but est de proposer un langage agnostique au matériel facilitant la mise à jour du traitement des paquets sans avoir à attendre l'aval des constructeurs qui, jusque là, décidaient des options rendues disponibles pour traiter les paquets. P4 a été mis à jour en 2016 pour créer P4₁₆, cette nouvelle version est plus modulaire et introduit une architecture normalisée.

1. *Simple Network Management Protocol* est un protocole de supervision de nœuds[FSDC90].

2. Le Tofino est un commutateur programmable, c'est un *ASIC* donc il est très efficace. La gamme Tofino 2 peut avoir 32 ports de 400Gb/s chacun, ce sont des commutateurs créés pour être inclus dans de grandes infrastructures.

3. Les FPGA sont des circuits électriques reconfigurables beaucoup moins efficaces que les *ASIC*. Leur programmabilité est utilisée pour prototyper des solutions dans plusieurs domaines, dont les réseaux par exemple.

4. Les NIC sont des machines possédant une partie CPU et une partie matérielle (possiblement programmable) interfacée avec la partie CPU. Le matériel programmable peut, par exemple, prendre la forme d'un FPGA.

3.3.1 Architecture

Les différentes architectures des équipements P4-programmables suivent une structure commune nommée *Protocol Independent Switch Architecture* ou PISA qui définit plusieurs éléments, en particulier (voir Figure 2) :

- Le *Parser* qui lit le paquet et en extrait les en-têtes ;
- La *Match Action Pipeline* qui est formée de *Match Action Tables* ou MAT. Les MAT sont des tables de correspondances entre des clés et des actions. Les clés sont, par exemple, des en-têtes ou des méta-données ;
- Le *Deparser* qui reforme un paquet pour l'émettre dans le réseau à partir des en-têtes et de la charge utile.

Chaque équipement peut implémenter sa propre version d'une chaîne de traitement PISA optimisée pour le type de trafic qu'il traitera. Par exemple, le Tofino compte 2 *Parsers*, 2 *Deparsers* et 2 *Match Action Pipelines*. L'architecture pour NetFPGA : SimpleSumSwitch [HHM⁺21], quant à elle, compte 1 *Parser*, 1 *Deparser* et 1 *Match Action Pipeline*.

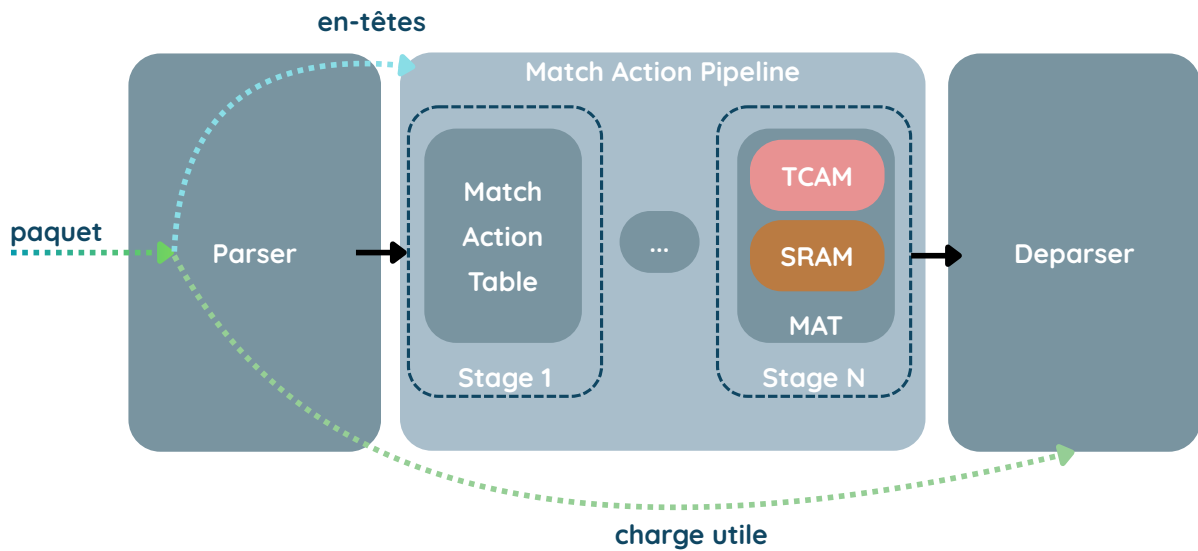


FIGURE 2 – Illustration de l'architecture et du fonctionnement d'un équipement P4-programmable dont le découpage en stage a été mis en avant ainsi que la présence de différents types de mémoires (*TCAM* et *SRAM* sont définis dans la section 3.3.2).

3.3.2 Fonctionnement

Le type de mémoire utilisé pour faire fonctionner les MAT peut également varier d'un équipement à l'autre, notamment en fonction de la clé utilisée. Comme dit précédemment, les MAT implémentent la logique du programme en déclenchant des actions suivant des clés. Par exemple, la commutation de niveau 2 serait définie par une clé représentant l'adresse de destination MAC et une action définissant le port de sortie.

La manière avec laquelle la variable doit correspondre à la clé peut varier. Le langage P4 définit trois types de correspondances :

- exactes : sert à faire correspondre deux variables de manière exacte bit à bit. L'exemple de commutation de niveau 2 utilisant l'adresse de destination MAC cité précédemment utiliserait une correspondance exacte.

- plus long préfixe (*lpm*) : correspond à la recherche du préfixe le plus long qui contient la valeur de la clé. Un exemple de *lpm* est présent dans la Figure 1 où l'adresse 10.0.0.42 est utilisée comme clé pour router le paquet vers le port 24. Ce port a été choisi plutôt que le port 8 car l'entrée avec le plus long préfixe dans ce cas est 10.0.0.0/24.
- ternaire : cherche une correspondance exacte sur certains bits. Les bits utilisés pour la correspondance sont définis par un masque binaire. Par exemple, si le masque n'est composé que de "1", alors tous les bits sont utilisés pour la correspondance et le résultat est équivalent à une correspondance exacte. De manière générale, la correspondance ternaire est équivalente à une opération ET bit à bit entre la clé et le masque suivie d'une correspondance exacte avec l'entrée dans la table.

Suivant la correspondance utilisée, le type de mémoire utilisée par la table peut varier. Dans les Tofinos, les correspondances exactes utilisent la *Static Random Memory Access* (SRAM) visible dans le stage N de la figure 2. La SRAM est le plus souvent utilisée en tandem avec des fonctions de hash pour chercher des correspondances exactes. C'est une mémoire plus efficace mais plus coûteuse en termes de prix et en termes de place physique sur le circuit que la *Dynamic RAM* retrouvée dans les ordinateurs personnels ou les serveurs.

Le *lpm* et la correspondance ternaire, quant à eux, utilisent la *Ternary Content-Addressable Memory* (TCAM) visible dans le stage N de la figure 2. La TCAM est une mémoire de type CAM, c'est-à-dire que chaque recherche est effectuée sur l'intégralité des cases mémoires allouées en même temps. Elle est particulièrement efficace pour les *lpm* car elle prend en compte un troisième caractère "*" nommé *wildcard* en plus des caractères binaires "0" et "1". La *wildcard* permet de faire la distinction entre le préfixe utilisant les caractères "0" et "1" (les bits identifiant un réseau) et le masque utilisant les caractères "*" (les bits identifiant les machines du réseau). Un exemple d'utilisation de TCAM pour une recherche dans un arbre à préfixes est présent dans les parties 2. et 3. de la Figure 1.

La *Match Action Pipeline* est découpée en plusieurs *stages* (voir Figure 2). Chaque *stage* possède une mémoire qui lui est propre, que ce soit de la TCAM, SRAM, ou une combinaison de plusieurs mémoires. Les MAT sont allouées dans un *stage*, ou plusieurs *stages* si elles nécessitent plus de mémoire que ce procure un seul *stage*.

Dans les Tofinos, les *stages* préparent un *Very Long Instruction Word* (VLIW). Ces VLIW peuvent être représentés comme une liste de toutes les variables d'un programme, et les actions qu'il faut appliquer à ces variables. La Figure 1 peut être utilisée comme exemple où, après avoir lu l'adresse IP de destination du paquet, le Tofino a préparé, dans un VLIW, que la valeur 24 allait être assignée à la variable responsable du port de sortie. Une fois le *stage* fini, le VLIW est exécuté et les changements sont appliqués sur toutes les variables en parallèle. Cette manière d'exécuter crée des cas de dépendances entre variables. Si deux MAT écrivent dans la même variable, par exemple le port de sortie, alors elles doivent non seulement définir un ordre de passage mais être allouées dans deux *stages* différents afin d'exécuter les écritures dans deux VLIW différents.

Les différents types de correspondances ne sont pas immuables, les constructeurs peuvent décider de rajouter de nouveaux types. Par exemple, les Tofinos possèdent un type de correspondance nommé *Algorithmic lpm* (*alpm*) qui fait l'objet d'un brevet [Wan]. Il est équivalent au *lpm* en termes de résultat, c'est-à-dire qu'il fait également une recherche du plus long préfixe correspondant, mais il utilise des optimisations pour réduire l'utilisation de la mémoire TCAM qui est présente en maigre quantité dans les Tofinos. L'*alpm* fonctionne en découpant l'arbre de recherche en plusieurs sous-arbres et en utilisant les mémoires SRAM et TCAM en tandem.

Alpm est une optimisation qui a été intégrée dans les Tofinos. Il existe d'autres méthodes d'optimisations dans la littérature, nous allons nous intéresser à celles qui réduisent la consommation.

4 L'étude de la consommation des réseaux : un état de l'art

Le *Green Networking* est une famille de sujets portant sur la consommation énergétique des réseaux. Il a été beaucoup étudié pendant les années 2010, en partie dû à l'essor du plan de contrôle programmable. Ces dernières années il redevient un sujet auquel la communauté scientifique s'intéresse, en partie dû à l'essor du plan de données programmable offrant de nouveaux angles d'approches mais aussi dû à la forte croissance du nombre de grandes infrastructures comme les centres de données (8000 en 2020 [Wis], 10 000 en 2025 [Sta]). L'*Internet Engineering Task Force* (IETF) a récemment créé un groupe de travail nommé *GREEN*¹ axé exclusivement sur la création de normes portant sur la consommation énergétique d'internet. La grande famille du *Green Networking* peut être décomposée de deux sous familles : l'optimisation et la prédiction.

4.1 Optimisation

L'optimisation de la consommation peut agir sur plusieurs éléments distincts dont le matériel visé, l'architecture dudit matériel et le contexte d'utilisation.

L'une des premières optimisations consiste à adapter le débit maximal supporté d'un émetteur-récepteur en fonction du trafic (*Adaptive Link Rate* (ALR)). Cette solution se base sur le fait que plus le débit maximal supporté d'un émetteur-récepteur est élevée, plus l'émetteur-récepteur consomme, peu importe la quantité de trafic le traversant. [JLV23] estiment que la moitié des émetteurs-récepteurs d'un centre de données peuvent être ajustés périodiquement et que ces modifications pourraient économiser plusieurs MWh par an. La principale limite à l'ALR est que le temps nécessaire à la transition entre deux capacités se compte en secondes voire minutes, nécessitant de pouvoir prédire le trafic. Bien qu'adapter le débit maximal supporté présente des défis de réactivité, l'introduction d'un mode "veille" est possible.

L'IEEE² a créé un standard nommé *Energy Efficient Ethernet* [ene10] qui passe très rapidement (quelques microsecondes) d'un mode veille à un mode actif. Le mode veille réduit la consommation de l'émetteur-récepteur lorsqu'il n'y a pas de trames à émettre comparé aux normes Ethernet sans mode veille. [BDB⁺11] et [RMHL12] ont étudié l'impact des deux technologies en tandem et jugent qu'il reste intéressant d'étudier ALR car elle permet de se rapprocher d'une consommation proportionnelle au taux d'utilisation. De manière générale, la majorité de la consommation énergétique des équipements réseau apparaît dès qu'ils sont allumés (alors qu'ils ne traitent pas de paquets)[Limb] [MHAT14]. Le traitement des paquets n'augmente que marginalement la consommation, c'est pourquoi certains vont jusqu'à éteindre les équipements.

Une autre approche consiste à éteindre des liens ou des nœuds. En effet, les réseaux sont surdimensionnés par rapport à la charge qu'ils doivent supporter. Dans le cas de [JLV23], la moyenne d'utilisation des liens est 18%. Lors de tels surdimensionnements, une technique plus extrême qu'ALR peut sembler viable : l'extinction de liens voire de nœuds.

Contrairement au mode veille de [ene10], l'extinction modifie la topologie logique du réseau et est donc plus compliquée à mettre en place même si elle économise plus d'énergie.

En 2023, [JV23] alertent que les réseaux filaires, en particulier ceux qui forment le cœur d'internet,

1. <https://datatracker.ietf.org/wg/green/about/>

2. L'*Institute of Electrical and Electronics Engineers* est un organisme de standardisation. <https://www.ieee.org/>

doivent s’inspirer des réseaux sans-fil et faire de la consommation d’énergie une métrique importante. Ils disent également que les équipements réseau doivent avoir une durée de vie plus longue en plus d’être éteints dès que possible afin de réduire les coûts énergétiques dû à la production et l’alimentation de tels équipements.

[GMMO10] montrent que le problème de recherche d’un graphe minimal capable de supporter des demandes (flux) dont le volume est connu à l’avance est un problème NP-complet. Ils montrent que, sur des topologies réelles (en ayant connaissance à l’avance des trafics) entre 10% et 50% des liens auraient pu être éteints sur toute la durée sans dégradation de la qualité de service.

[RJV24] ont développé une heuristique pour éteindre des liens de manière dynamique, sans avoir connaissance des trafics en avance. Leurs simulations éteignent environ 30% des liens dans des réseaux où la charge moyenne est entre 1% et 2%. Les réseaux étudiés sont des *National research and education network*, signifiant que leur charge moyenne est plus basse que celle de réseaux d’opérateurs privés.

Certains articles vont plus loin que l’extinction de liens et éteignent des nœuds. *P4Green* se concentre sur un réseau de centre de données où, sans l’aide d’un contrôleur central, elle arrive à éteindre 36% des commutateurs en moyenne [GK23]. Cette solution n’éteint que les *switchs d’agrégation* qui sont nombreux car ils servent à réduire la congestion entre les serveurs du centre de données et le réseau externe. Il est intéressant de noter que P4Green utilise le langage P4 pour rediriger dynamiquement le trafic vers des serveurs alimentés en énergies renouvelables.

Il est également possible de réduire l’utilisation de ressources en optimisant le code P4 directement. Les optimisations se basant sur le langage P4 sont nombreuses car elle sont automatiquement déployable sur tous les équipements P4-programmables. À ma connaissance, il n’existe pas d’étude portant spécifiquement sur la réduction de la consommation énergétique de commutateurs via des optimisations de leur code P4. Cela ne veut pas dire que ces optimisations ne réduisent pas la consommation énergétique des commutateurs, seulement que leur impact n’a pas été étudié. Il reste donc intéressant de s’intéresser à ces optimisations car certaines d’entre elles sont possiblement des sources de réduction de la consommation.

[WLJ⁺24], par exemple, regroupe les différentes politiques du plan de contrôle (quels préfixes doivent être routés en direction de quel port, quelles associations pour le NAT, MPLS, *etc.*) et les découpe en plusieurs MAT qu’ils arrangent de telle sorte à ce que le nombre de *stages* soit minimal.

[CLX⁺23] décrivent un outil qui réduit l’utilisation d’un type de mémoire très précis : le *Packet Header Vector* (PHV). Les PHV sont des blocs mémoire présents dans les Tofinos qui contiennent les en-têtes et méta-données qui sont traitées le long de la *Match Action Pipeline*.

Cetus [LGZ⁺22] est un outil cherchant à réduire le nombre de stages nécessaires pour traduire un code P4 en un programme alloué en mémoire. Pour rappel, dans un programme P4, un nouveau stage est alloué si la quantité de mémoire n’est pas suffisante ou si des dépendances sont présentes. Cetus résout certaines dépendances pour réduire le nombre de stages.

[WADV20] propose, comme Cetus, de résoudre des dépendances pour réduire le nombre de stages mais aussi de réduire la taille de certaines tables et de déléguer certaines fonctionnalités au plan de contrôle (en particulier celles qui n’apparaissent que très peu souvent). Ces optimisations sont déduites en prenant une trace de paquets censée représenter le trafic. Chaque passe d’optimisation est validée à l’aide de la trace, afin de ne pas créer d’incohérences de traitement entre le programme sans optimisations et le programme avec optimisations.

Pipeleon [XQH⁺23] est un outil spécialisé pour les SmartNIC. L’outil crée plusieurs caches, ré-ordonne les tables et fusionne des tables entre elles. Il utilise un modèle de prédiction de la consommation pour estimer l’impact des optimisations proposées et choisit l’ensemble d’optimisations les plus impactantes.

Enfin, la consommation peut être réduite via l’utilisation d’un cache. Par exemple, les auteurs de [LVCG05] décrivent un *Computation Reuse Cache* visant à réduire la consommation en conservant en mémoire le résultat de traitements coûteux en énergie. Ils s’intéressent principalement à un principe de

localité de l'adresse IP de destination, entre autres, c'est-à-dire que plusieurs flux différents partagent une même adresse de destination.

Les limites du *Computation Reuse Cache* que j'aborde durant ce stage sont l'absence de choix des entrées qui peuplent le cache, toutes les entrées sont ajoutées au cache, et l'absence d'étude des problématiques que peuvent engendrer le cache (*e.g.* incohérence après avoir mis à jour une table de routage sans avoir mis à jour le cache).

4.2 Prédiction

La prédiction de la consommation énergétique d'équipements réseau dépend, en partie, du débit entrant et de paramètres propres à chaque commutateur ou routeur. [Limb], comme [SVZR11] et [MHAT14], utilise une modélisation linéaire pour estimer la consommation qui peut être formalisée à l'aide des paramètres et données en entrée suivants :

- $P_{inactif}$: la consommation lorsque l'équipement est branché mais inactif, qui représente la majorité de la consommation totale ;
- D_{paquet} : le nombre de paquets qui traversent l'équipement par seconde ;
- L : la taille des paquets qui traversent l'équipement ;
- E_{paquet} : la consommation de l'équipement par paquet, c'est-à-dire la quantité d'énergie qu'il faut pour lire les en-têtes, les utiliser pour router, écrire des labels, *etc.* ;
- E_{bit} : la consommation de l'équipement par bit, c'est-à-dire la quantité d'énergie qu'il faut pour faire passer la charge utile : les données sans les en-têtes, du port d'entrée au port de sortie.

Une version graphique du modèle est disponible dans la Figure 3.

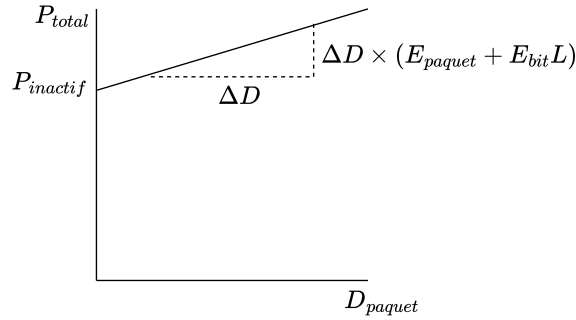


FIGURE 3 – Modèle d'estimation de la consommation tel que décrit dans l'état de l'art.

Ce modèle est cependant incomplet si l'on considère des équipements programmables. En effet, la consommation évolue en fonction de la configuration des ports mais aussi en fonction du programme P4 qui est exécuté dans le Tofino [Lima]. Ces résultats sont une fondation importante de ce stage car ils montrent que la consommation ne dépend pas uniquement des paramètres cités dans le modèle précédent.

4.3 Taxonomie

L'ensemble des leviers ayant une influence sur la consommation énergétique est visible dans la taxonomie illustrée dans la Figure ??.

Ici, trois catégories apparaissent : les réseaux non-programmables, les réseaux dont le plan de contrôle est programmable (SDN) et les réseaux possédant des équipements dont le plan de données est programmable. Le plus souvent, les leviers qui partagent la même catégorie pourront être étudiés en tandem.

Le second axe qui a été choisi pour trier est la dépendance au matériel ou à l'architecture. Par exemple, P4 est un langage de haut niveau déployable sur plusieurs cibles. Si une optimisation est applicable au langage P4 alors elle sera applicable à l'ensemble des équipements compatibles. Dans le cas contraire,

si une optimisation ne s'applique qu'à une topologie de centre de données, alors elle devient fortement contrainte et n'est pas applicable à d'autres topologies.

Pendant ce stage, je me suis particulièrement intéressé aux éléments de la colonne "Nécessite la programmabilité du plan de données", dont l'utilisation des mémoires TCAM/SRAM, le nombre d'entrées et le nombre de stages.

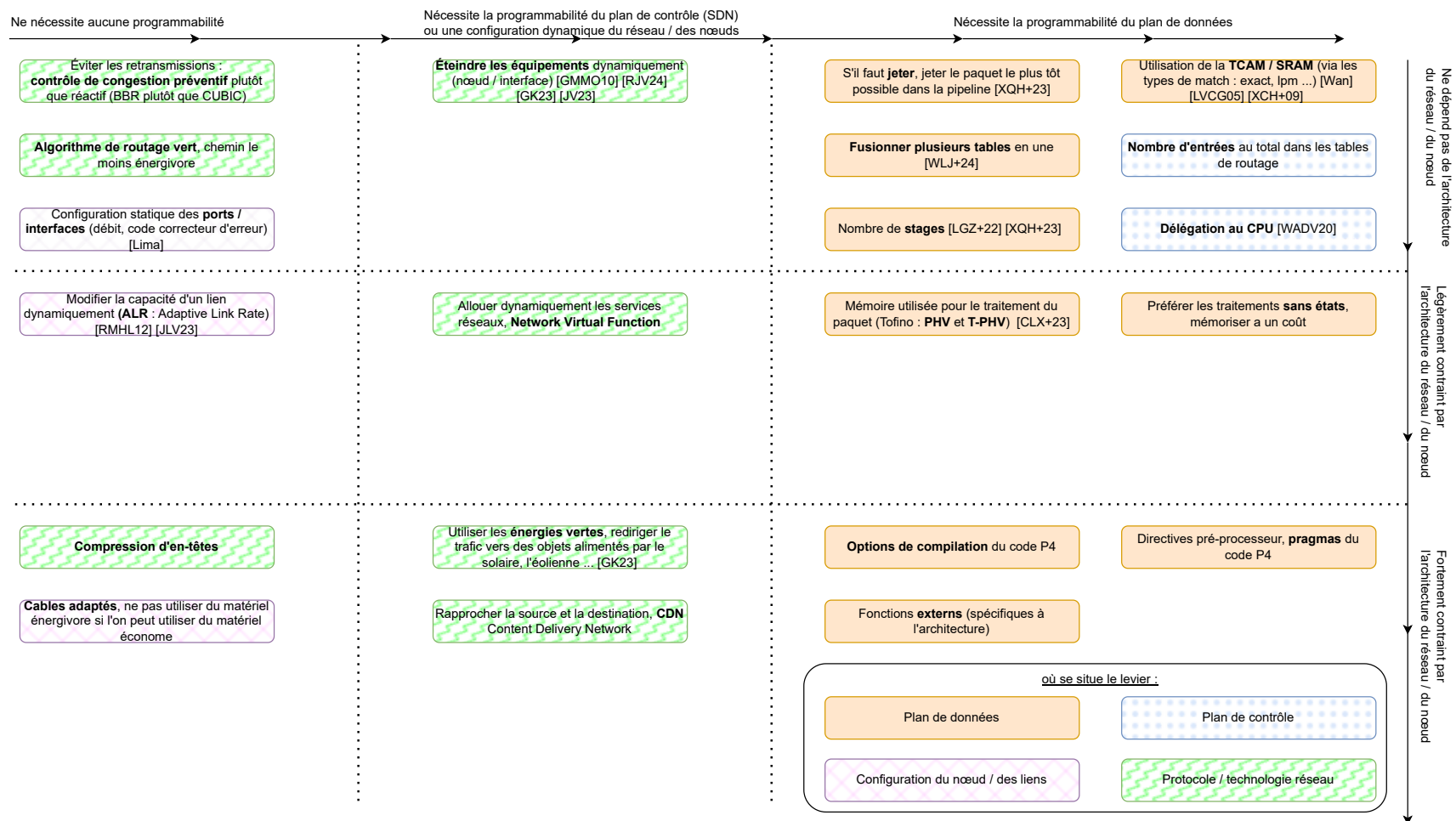


FIGURE 4 – Taxonomie des leviers qui agissent sur la consommation énergétique des réseaux.

5 Préparation à la modélisation

Les modèles d'estimation de la consommation énergétique sont des outils qui utilisent des données en entrée et des paramètres pour estimer la consommation d'un système. Le résultat étant une consommation énergétique en Joules ou une puissance en Watts (1 Watt = 1 Joule / 1 Seconde).

Un modèle de consommation d'un équipement est un outil très intéressant, en particulier pour estimer l'impact d'une optimisation, voire même pour valider ladite optimisation. Par exemple, l'outil d'optimisation Pipeleon [XQH⁺23] propose des optimisations et fait passer chacune d'entre elles dans un modèle pour savoir lesquelles sont les plus performantes.

Dans mon cas, le modèle sert principalement à trier les paramètres en fonction de leur impact dans la consommation d'énergie, car ceux qui sont jugés comme étant les plus énergivores seront les plus intéressants à optimiser. Mon objectif final est de développer des méthodes d'optimisations de la consommation énergétique des réseaux programmables, en particulier des équipements P4-programmables.

Il est important de souligner que toutes les estimations de la consommation énergétique utilisées sont le fruit du compilateur Intel. L'intention initiale était d'utiliser les équipements de la plateforme *Grid5000* de *SlicesFR*, mais ils n'ont malheureusement pas été prêts à temps.

5.1 La plateforme *SlicesFR*

SlicesFR est une infrastructure de recherche contenant des machines auxquelles des chercheur-e-s peuvent accéder pour faire des expériences. Parmi ces machines se trouvent des objets communicants, des serveurs pour effectuer des calculs distribués ou intensifs, *etc.* L'infrastructure héberge également des Tofinos, la gamme P4-programmables de commutateurs d'Intel. Au début de mon stage nous (Maxime FRIESS, camarade stagiaire, et moi) avons obtenu l'autorisation d'accéder à ces machines qui n'avaient encore jamais été utilisées dans le contexte de la plateforme *Grid5000*¹ de *SlicesFR*.

Ayant déjà de l'expérience avec P4 et le simulateur de commutateur d'Intel, Maxime et moi avons participé à l'intégration des Tofinos dans la plateforme avec l'aide des ingénieurs responsables. Nous avons réussi à installer le logiciel permettant le déploiement de code P4 dans le matériel. J'ai aussi rédigé une procédure² pour initialiser les interfaces CPU, les ports du commutateur, les tables dans le programme P4 et émettre des paquets qui transitent de commutateur en commutateur. Cette procédure a été mise à disposition des ingénieurs de la plateforme et devrait aider à la création d'un tutoriel expliquant le fonctionnement des Tofinos.

Les Tofinos sont équipés de wattmètres indiquant la puissance énergétique de ceux-ci. Malheureusement, ces données n'ont pas été rendues accessibles à temps et je n'ai pas pu faire d'expériences pour récupérer la puissance énergétique réelle des Tofinos. J'ai donc utilisé les estimations de la puissance énergétique que donne le compilateur Intel pour former un squelette dans lequel il suffirait de remplacer la puissance estimée par la puissance réelle pour avoir un jeu de données qui reflète la réalité.

Les études existantes [FOZZ⁺24] [Lima] proposent déjà des modèles et des jeux de données mais les informations dans ces jeux de données ne sont pas suffisantes pour trier les paramètres responsables de la consommation en énergie avec la granularité souhaitée.

1. www.grid5000.fr

2. <https://filesender.renater.fr/?s=download&token=b86b048f-adce-49d8-b0db-3cdf9b1f1095> : FileSender est un service hébergé par *RENATER*.

5.2 Observations préliminaires

L'outil d'estimation de la puissance énergétique d'Intel est une boîte noire ; il n'est pas possible de savoir quels critères du programme sont jugés importants. On sait, néanmoins, que le compilateur suppose le pire scénario possible, en termes de puissance énergétique : tous les paquets sont supposés très petits, maximisant donc le nombre de paquets par seconde. Il suppose aussi que chacun des paquets subit le traitement le plus complexe possible (*i.e.* le chemin d'exécution le plus long). Suivant ces hypothèses, le compilateur fournit une unique valeur, en Watts, par programme P4.

La Figure 4 montre des exemples d'estimations du compilateur en fonction du nombre d'entrées dans deux tables : la première utilise l'*alpm* et la seconde le *lpm*. Pour obtenir ces résultats, j'ai créé plusieurs codes P4 qui n'utilisent qu'une seule table dont j'ai fait varier la taille maximale. Étant donné que le compilateur estime le pire cas possible, la taille maximale est jugée comme la taille effective des tables. On remarque que la puissance commence à 0W et non à $P_{inactif} \approx 130W$ [Lima] car le compilateur n'estime que le surplus de puissance de la part des chaînes de traitements. On remarque également que la puissance évolue différemment en fonction de deux paramètres : le type de correspondance et le nombre d'entrées dans les tables. Plus il y a d'entrées dans les tables, c'est-à-dire plus on utilise de mémoire, plus le commutateur consomme de l'énergie. De même la puissance évolue différemment en fonction des types de correspondances : l'*alpm* consomme moins que *lpm* pour des grands nombres d'entrées. Cette puissance n'est pas négligeable non plus, elle atteint les 35W soit 26% de la puissance $P_{inactif}$.

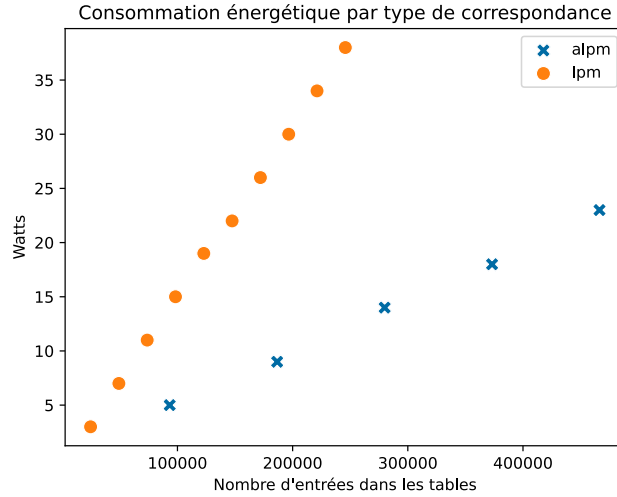


FIGURE 5 – Estimations de la puissance par le compilateur Intel en fonction de la quantité de mémoire allouée et du type de correspondance.

Ces deux paramètres (le type de correspondance et le nombre d'entrées) n'apparaissent pas dans les modèles de l'état de l'art. Il est néanmoins possible que ces paramètres soient considérés implicitement dans le paramètre E_{paquet} . Malgré cela, ce paramètre est sensé être universel pour tous les commutateurs de la même famille, or ici il dépend du nombre d'entrées et du type de correspondance. Il faudrait alors le remplacer par une fonction du type : $E_{paquet}(match_type, table_size)$, où, par exemple, pour $N > 10000$, $E_{paquet}(lpm, N) > E_{paquet}(alpm, N)$.

Il est aussi intéressant de noter qu'une case mémoire *lpm* consomme environ 30 fois plus qu'une case mémoire correspondance exacte. Cette importante différence est due au fait que *lpm* utilise la mémoire TCAM qui est très coûteuse en énergie, comparée à la mémoire SRAM utilisée par la correspondance exacte.

Il devient alors légitime de se demander quels autres leviers il faut prendre en compte, quelle importance ont-ils, et comment faire pour modéliser ces nouveaux paramètres.

5.3 Méthodologie

Pour identifier ces nouveaux paramètres, j’ai d’étudié les spécifications du langage P4 et des Tofinos afin d’en extraire le plus de leviers possibles. Parmi ces leviers se trouvent, comme mentionné précédemment, la taille des tables, le nombre de stages, entre autres. Une fois la liste de leviers dressée (20 au total), j’ai écrit un script qui génère des codes P4 en faisant varier indépendamment un, ou plusieurs, leviers.

Ainsi, j’ai généré 585 codes P4 et compilé chacun d’entre eux afin d’en extraire les informations supplémentaires du compilateur comme le nombre de blocs mémoire TCAM alloués, le nombre de *stages* en plus de l’estimation de la puissance. À l’aide de ces informations, j’ai créé un jeu de données qui liste tous les leviers qui ont été mis en avant pour chaque programme, les informations procurées par le compilateur et la puissance énergétique estimée, soit environ 100 paramètres au total.

Les outils statistiques choisis pour analyser ces données et retrouver les leviers responsables de la consommation sont la corrélation et la *Permutation Feature Importance* [FRD19] (PFI). Selon la Figure 5a, la puissance énergétique est fortement corrélée à l’utilisation de la mémoire TCAM, mais aussi en partie la mémoire SRAM et la latence. La corrélation est un outil intéressant pour filtrer les nombreux paramètres, en effet parmi les 100 paramètres récoltés, seuls 30 environ avaient un coefficient de corrélation supérieur à 0.1 ou inférieur à -0.1 signifiant qu’aucun lien significatif n’est présent entre ces paramètres et la puissance estimée.

Avec l’aide d’un modèle de prédiction, la PFI modifie les paramètres de ce dernier pour voir à quel point le résultat en est impacté. La PFI en déduit alors l’importance de chaque paramètre. PFI permet de capturer des relations non-linéaire dans un modèle entraîné sur plusieurs paramètres, ce qui n’est pas possible avec la corrélation. Par exemple, sur un modèle *random forest* entraîné sur le jeu de données qui utilise l’estimation du compilateur, la PFI juge que l’utilisation de la TCAM est, de loin, le facteur le plus important lorsqu’il s’agit d’estimer la puissance (voir Figure 5b). Elle trouve aussi que la taille des tables, une option de compilation qui réduit le trafic maximal accepté, l’utilisation de la SRAM et la latence sont importants.

Le code utilisé pour générer les codes P4, les compiler et les analyser est présent dans l’archive ³.

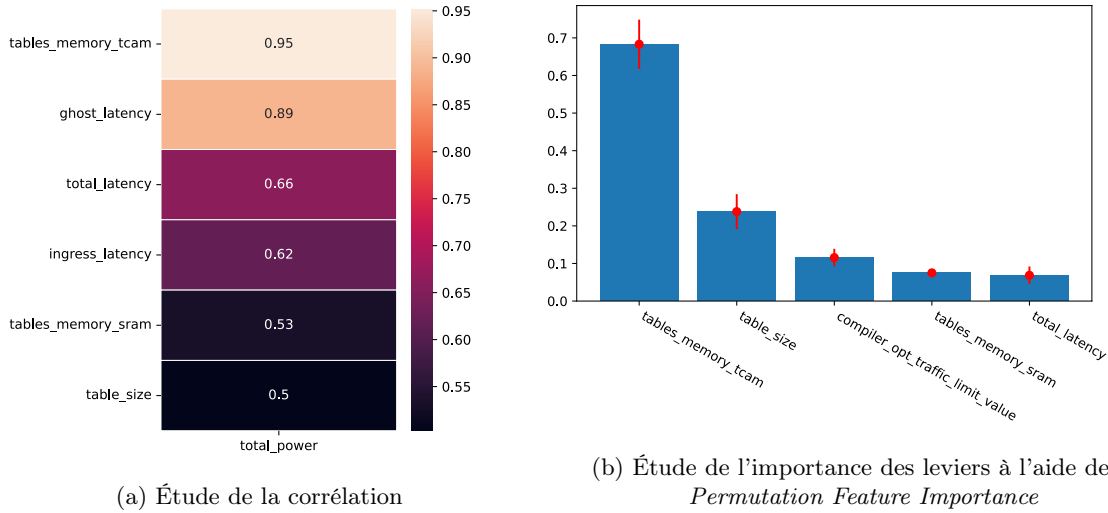


FIGURE 6 – Résultats de la recherche des leviers les plus importants selon (a) la corrélation (b) la *Permutation Feature Importance*

Étant donné que l’utilisation de la mémoire TCAM est le levier le plus important, les optimisations présentées dans le chapitre suivant se focalisent sur l’utilisation de cette mémoire.

3. <https://filesender.renater.fr/?s=download&token=e3bf535a-0bd1-4b24-9629-a172e434092f> : *FileSender* est un service hébergé par *RENATER*.

6 Préparation à l'optimisation

Dans le chapitre précédent, j'ai montré que l'utilisation de la mémoire TCAM constitue le paramètre ayant le plus d'influence sur la consommation. Dans ce chapitre je présente des solutions pour en réduire l'utilisation. La TCAM est utilisée pour effectuer des *lpm* qui sont non seulement important mais aussi très fréquents dans les réseaux. Les optimisations proposées sont (1) le dépliage de préfixe, consistant à transformer une entrée *lpm* en plusieurs entrées à correspondance exacte équivalentes. (2) Le découpage puis ordonnancement des tables qui nécessitent plusieurs stages. Les sous-tables sont triées par ordre de popularité, c'est-à-dire que les préfixes qui sont le plus souvent utilisés sont mis en premiers, et les moins utilisés sont mis à la fin de la chaîne de traitement. (3) Un cache énergétique qui garde en mémoire les correspondances entre adresse IP de destination et port de sortie. Ce cache utilise la correspondance exacte et est placé avant une table *lpm*. Si l'adresse de destination est présente en cache alors il n'y a pas besoin de faire la recherche dans la table *lpm* (qui utilise la mémoire TCAM).

Ces optimisations reposent sur les hypothèses suivantes :

- L'utilisation des mémoires (TCAM et SRAM) a un impact non-négligeable sur la consommation énergétique du commutateur ;
- Une entrée dans une table (MAT), telle que définie dans le chapitre 3, consomme particulièrement plus si elle utilise le *lpm* que si elle utilisait une correspondance exacte ;
- Une table (MAT) non sollicitée lors du traitement d'un paquet consomme moins d'énergie qu'une table effectivement utilisée.

6.1 Dépliage de préfixe

Comme vu précédemment, la correspondance exacte, utilisant la mémoire SRAM, consomme moins d'énergie par case mémoire que le *lpm*, utilisant la mémoire TCAM. L'idée de déplier certains préfixes permet de retirer des entrées des tables *lpm* pour les mettre dans une nouvelle table à correspondance exacte.

Les préfixes de taille /27 correspondent à un réseau de taille $2^{32-27} = 32$, soit légèrement plus que le ratio entre la consommation d'une case mémoire *lpm* et une case mémoire à correspondance exacte (cf résultats présents dans le chapitre 3). Il faut alors prendre les entrées *lpm* dont la longueur de préfixe est entre /28 et /32, pour énumérer toutes leurs adresses dans une table à correspondance exacte afin de réduire la consommation. Sur la Figure 6, on voit une entrée 10.0.0.0/31 dans la table *lpm* devenir les deux entrées 10.0.0.0 et 10.0.0.1 dans la table à correspondance exacte. Les paquets correspondant à ces entrées n'ont pas besoin d'être traités par la table *lpm*. Dans cet exemple, la consommation liée à ce préfixe est $\frac{2}{30} = 6.67\%$ ce qu'elle était sans le dépliage.

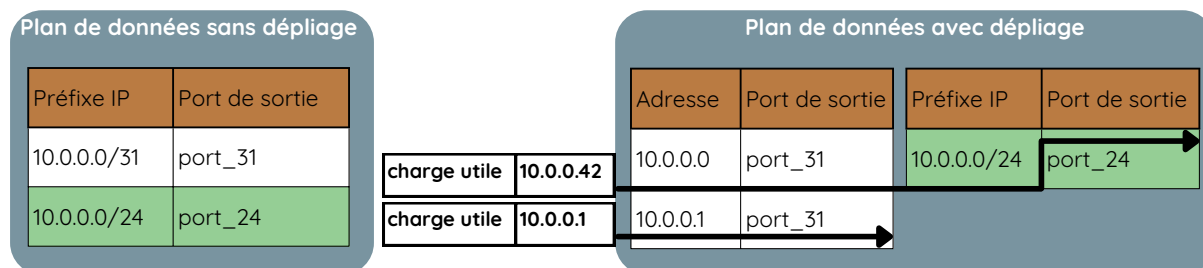


FIGURE 7 – Illustration d'un plan de données avant et après le dépliage de préfixes.

Lorsqu'on remplace un *lpm* par une correspondance exacte, il est important de prendre en compte que plusieurs préfixes peuvent correspondre à une adresse donnée. Par exemple, l'adresse 10.0.0.0 fait partie des préfixes 10.0.0.0/8 et 10.0.0.0/24, pourtant c'est l'entrée avec le plus long préfixe, ici 10.0.0.0/24 qui sera choisie. Dans notre cas, il suffit de déplier les préfixes en fonction de leur longueur. Le préfixe le moins long est déplié en premier, ensuite le préfixe plus long écrasera l'entrée précédente. Il est nécessaire de faire ce travail en amont, et non directement dans les tables de routage, afin qu'aucun paquet ne soit incorrectement routé pendant le dépliage de plusieurs préfixes.

Une nouvelle table à correspondance exacte visant à réduire la consommation doit être placée dans un stage avant la table *lpm* dont on souhaite éviter l'utilisation (voir Figure 6). Cette nouvelle table crée un surplus de consommation qui ne compense pas nécessairement les gains liés au dépliage de préfixe. L'efficacité de cette optimisation dépend de l'énergie dépensée par les tables *lpm* et à correspondance exacte, et du trafic. Si le trafic est routé uniquement à l'aide des entrées dans la table à correspondance exacte alors on observera une réduction de la consommation comparé au programme sans dépliage de préfixe.

Dans un routeur, la majorité des informations concernent des destinations externes au réseau. Pour des raisons de passage à l'échelle, la longueur des préfixes des annonces externes est au plus /24. Cette optimisation ne fonctionne donc pas sur les préfixes externes mais elle peut être utilisée sur les destinations internes, qui sont très peu nombreuses comparées au nombre de préfixes externes ($\approx 10^6$).

Afin de correspondre aux routeurs de bordure, j'ai proposé l'optimisation suivante cherchant à réduire la consommation des commutateur ayant beaucoup de préfixes à router.

6.2 Modifier l'ordonnancement des tables

Le ré-ordonnancement de tables est une technique visant à réduire la durée moyenne des traitement des paquets [XQH⁺23]. Si l'on découpe et réordonne de grandes tables, il devrait être possible de réduire la consommation en plus de réduire le temps de traitement des paquets.

L'idée fonctionne sur des grandes tables qui ont besoin d'allouer la mémoire de plusieurs stages, qu'elles soient *lpm* ou à correspondance exacte. L'optimisation consiste à découper une grande table qui s'étend sur plusieurs stages en plusieurs sous-tables qui s'étendent sur un seul stage, de cette manière on peut agencer les préfixes dans les sous-tables par ordre de popularité, c'est-à-dire le nombre de fois qu'ils sont accédés. Les préfixes qui sont le plus souvent accédés seront placés dans la sous-table au début de la chaîne de traitement (indice N) alors que ceux qui sont moins souvent accédés iront dans la sous-table qui a été placés derrière la sous-table contenant les préfixes populaires (indice $M > N$).

Il faut néanmoins faire attention au *lpm*, pour lequel il faut ajouter de la nuance. On rappelle que plusieurs entrées/préfixes peuvent correspondre à une adresse donnée, si l'on coupe la table en plusieurs morceaux et que des préfixes se retrouvent dans des tables différentes, il se peut que des erreurs de routage apparaissent. Par exemple si les deux préfixes 10.0.0.0/8 -> `port_8` et 10.0.0.0/24 -> `port_24` se retrouvent dans le stage 1 et 2 respectivement, alors un paquet à destination de 10.0.0.42 sera routé en direction du `port_8` de manière erronée. Pour palier ce problème, il faut ajouter les préfixes qui incluent, ou sont inclus, dans les préfixes populaires aux tables dans lesquelles sont attribuées les préfixes populaires.

Plutôt que d'avoir une table immuable et statique, je propose une table qui s'adapte à la majorité des flux retrouvés sur Internet.

6.3 Cache énergétique

À ma connaissance, l'idée d'un cache utilisé pour réduire la consommation n'a pas reçu d'attention depuis 2005 [LVCG05]. Dans ce document, le cache sera étudié dans le cadre d'un ToFino mais l'idée peut être généralisée sur différents types d'équipements réseau.

Le cache énergétique prend la forme d'une table à correspondance exacte (voir Figure 7) qui garde en mémoire certaines associations entre adresse IP de destination et port de sortie de la table *lpm*, en particulier celles dont on sait qu'elles sont utilisées pour router des flux volumineux. On dira que le cache "recrute" les entrées de la table *lpm*. Le but étant de réduire la consommation liée au traitement de paquets qui partagent la même adresse de destination, typiquement les paquets qui appartiennent au même flux.

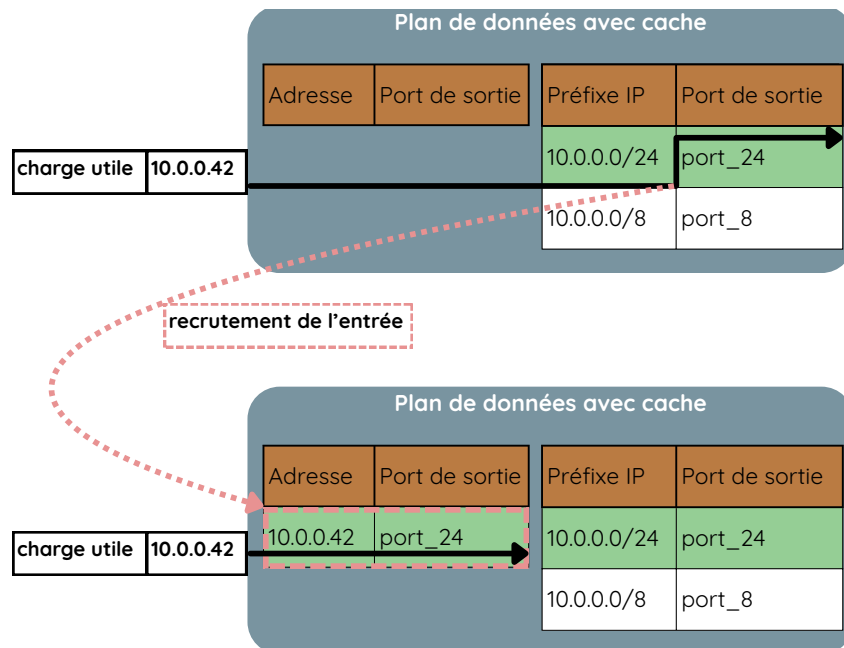


FIGURE 8 – Illustration du fonctionnement du cache et du recrutement des entrées.

Il se trouve que les flux volumineux représentent la grande majorité du volume de trafic qui transite sur internet. Selon [San], 80% du volume du trafic provient du streaming ou des partages de fichiers. Le but du cache énergétique est d'utiliser cette propriété pour réduire la consommation énergétique des commutateurs.

Ce trafic va souvent dans la même direction, approximativement 50% du volume de trafic total provient des 5 mêmes sources de streaming : *Youtube*, *Netflix*, *TikTok*, *Disney+* et *Amazon Prime* [San]. Le cache pourrait donc être réutilisé par plusieurs flux voulant communiquer avec les mêmes destinations, qui sont évidemment plus nombreuses que 5, chaque service ayant plusieurs serveurs (par exemple des *Content Delivery Networks*¹). Pourtant, il est fort probable que plusieurs, voire beaucoup, de flux puissent bénéficier d'une seule et même entrée dans le cache².

De plus, dans les réseaux de cœurs modernes, le calcul du port de sortie (pour les paquets dont la destination est à l'extérieur du réseau) consiste en une recherche *lpm* pour trouver le routeur de bordure auquel on doit envoyer le paquet ; suivi d'une recherche *lpm* dans la table de routage interne afin de router le paquet en direction du prochain routeur interne, c'est-à-dire le routeur suivant dans le plus court chemin reliant le routeur courant et le routeur de bordure [Jun] [Cis]. Tous ces traitements, ainsi que d'autres comme l'écriture de labels MPLS ou la correspondance NAT, peuvent être conservés en cache [LVCG05] [ZBZ⁺18].

1. Les CDN ou *Content Delivery Network* sont un moyen de rapprocher le contenu et l'utilisateur-riche qui le souhaite. Des films, séries, fichiers, etc. sont stockés dans des serveurs, et l'utilisateur-riche est redirigé-e vers le serveur le plus proche qui possède la ressource qu'il souhaite.

2. Malheureusement, la majorité du volume provient de ces services et va en direction des utilisateur-riche-s. La mutualisation des adresses de destination dans le cache énergétique n'aurait pas d'impact sur ces énormes flux qui transportent des vidéos mais plutôt sur les acquittements, requêtes HTTP, etc. envoyés en direction de ces services.

6.3.1 Proposition d'implémentation déterministe

Cette implémentation alloue des compteurs qui servent à compter le nombre de paquets qui utilisent la même adresse de destination. Les adresses qui ont été le plus sollicitées sont recrutées dans le cache énergétique. Les entrées qui ont été recrutées en cache ont un temporisateur qui, lorsqu'il est expiré, réinitialise le compteur pour cette adresse de destination et libère la place de l'entrée dans le cache. Cela permet de cibler les destinations qui ont été récemment demandées et de rendre le cache dynamique.

Les *sketches* sont des manières d'obtenir de l'information concernant les flux en utilisant des méthodes d'analyse statistiques [ZCQ⁺23]. Ils prennent la forme de fenêtres glissantes des N derniers paquets ou des T dernières secondes de trafic et utilisent des méthodes d'optimisation de mémoire (allocation dynamique de bits, réutilisation de mémoire). Cette forme de collecte permettrait de réduire la quantité de ressources nécessaires pour compter les paquets pour les différentes adresses.

Le principal point positif de cette implémentation est qu'elle est déterministe et dynamique, les entrées dans le cache énergétique appartiennent forcément à un flux volumineux. Le principal point négatif de cette implémentation est qu'elle nécessite des ressources en plus du cache énergétique, les compteurs ont un coût non-négligeable qui n'est probablement pas nécessaire.

6.3.2 Proposition d'implémentation probabiliste

Étant donné que les flux ciblés représentent une majorité du volume de trafic qui traversera le commutateur, il est légitime d'utiliser une méthode probabiliste qui pioche au hasard des paquets dont l'adresse de destination est recrutée en cache. Ces paquets ont naturellement une forte probabilité d'appartenir à un flux volumineux, l'entrée recrutée a donc une forte probabilité d'être réutilisée. Le cache utiliserait une politique de tourniquet, c'est-à-dire une file où s'il n'y a plus de place, les nouvelles entrées chassent les entrées les plus anciennes.

L'utilisation de cette définition d'ancienneté implique qu'il y aura un renouvellement constant des entrées présentes dans le cache. En effet, s'il y a un plus grand nombre de flux que de place dans le cache, alors les flux qui ne sont pas dans le cache vont, au bout d'un moment être recrutés et chasser les anciennes entrées du cache. Ces entrées qui viennent d'être chassées font désormais partie des flux qui n'utilisent pas le cache et vont, à leur tour, être recrutées et chasser une entrée.

Supposons qu'il y ait un recrutement toutes les secondes dans un cache capable de supporter 10 entrées. Un "mauvais recrutement", c'est-à-dire un recrutement qui ne serait jamais utilisé par aucun flux, passerait environ 10 secondes dans le cache énergétique. Il occuperait la place d'une entrée qui serait potentiellement plus utile car plus souvent sollicitée.

Pour atténuer cet effet, il suffit d'ajouter des temporisateurs aux entrées du cache et de réinitialiser ces temporisateurs à chaque fois que l'entrée est sollicitée (*Least Recently Used* (LRU) [OOW93]). Cette méthode vise à remplacer la notion d'ancienneté d'ajout par une notion d'ancienneté d'accès. Prenons l'exemple d'un cache supportant 10 entrées, où 9 d'entre elles sont actuellement utilisées par des "bons recrutements" (leurs temporisateurs contiennent une valeur faible car ces entrées sont régulièrement accédées). Si la 10^{ième} place est occupée par un "mauvais recrutement", l'entrée verra son temporisateur augmenter au fil du temps jusqu'à ce qu'il dépasse celui des 9 autres "bons recrutements". À ce moment, l'entrée deviendra la plus ancienne en termes de temps d'accès et sera la première à être chassée.

LRU n'est pas la seule manière de réduire le nombre de "mauvais recrutements", il existe par exemple l'algorithme de la seconde chance. Cet algorithme alloue un bit à chaque entrée, si l'entrée est sollicitée, alors on lui alloue une seconde chance, représentée par la valeur "1". Si l'entrée n'est pas sollicitée, son bit vaut "0" et elle devient candidate pour être chassée du cache. Il existe plusieurs améliorations de l'algorithme de la seconde chance et de LRU. Par exemple, [JS94] définit deux listes LRU, la première, dite "lente" recrute les entrées de la table *lpm* alors que la seconde, dite "rapide", recrute les entrées du premier cache LRU. La chaîne ira lire dans le cache "rapide" en premier, puis dans le cache "lent" pour finalement aller chercher dans la table *lpm*.

Comparativement à [LVC05], la solution probabiliste présente les différences suivantes : un choix intelligent des entrées, elles ne sont pas naïvement mises à jour pour chaque paquet, il y a un choix déterministe ou probabiliste s'appuyant sur des observations quant aux propriétés des flux. Elle présente également une manière d'atténuer la durée dans le temps des "mauvais recrutements".

6.3.3 Analyse numérique de l'impact d'un cache probabiliste

Une analyse numérique sert à quantifier l'impact d'une telle solution, dans notre cas l'étude se base sur des estimations de la consommation, le résultat sera donc une estimation de l'impact. Le principal résultat est que le surplus de consommation dû à l'ajout d'une nouvelle table est compensé par l'énergie économisée si le nombre de paquets est suffisant.

Selon [FOZ⁺24], le traitement d'un paquet dans un programme P4 dure environ (en nanosecondes) :

$$143.441 + 0.056 \times p + \left(\sum_1^n 0.105 \times hs_n + 1.431 \right) + 0.607 \times t_e + 0.685 \times t_{e'} + 0.749 \times t_l + 0.896 \times t_{l'}$$

où

- p la taille du paquet
- hs_n est la taille du n^{ieme} en-tête lu dans le parser pour chaque état du parser ;
- t_e est le nombre de tables à correspondance exacte dont la clé est de taille 8 bits ;
- $t_{e'}$ est le nombre de tables à correspondance exacte dont la clé est de taille 16 ou 32 bits ;
- t_l est le nombre de tables lpm dont la clé est de taille 8 bits ;
- $t_{l'}$ est le nombre de tables lpm dont la clé est de taille 16 ou 32 bits.

Dans notre cas, sans le cache énergétique, c'est-à-dire uniquement avec une table lpm :

$$143.441 + 84 + (2.901 + 3.531) + 0.896 \times 1 = \boxed{234.769 \text{ nanosecondes}}$$

Le compilateur nous informe que, dans le pire cas, le code sans le cache nécessite 225 tics d'horloge et celui avec le cache : 266, soit 1.182 fois plus long. L'estimation de [FOZ⁺24] ne prend pas en compte la création du message en direction du plan de contrôle qui recrute une entrée au cache. Avec l'estimation de [FOZ⁺24], le programme est jugé 1.003 fois plus long. Le facteur 1.182 est donc utilisé pour maximiser les chances du programme sans cache.

Le compilateur nous informe également du coût de l'initialisation de la première case mémoire, et de celles qui suivent :

	$alpm$	lpm	exacte
Coût de la première case mémoire	0.54	0.16	0.21
Coût des cases suivantes	48×10^{-6}	155×10^{-6}	5×10^{-6}

Donc dans le cas où nous n'utilisons qu'une table lpm de taille 10^6 (représentant approximativement le nombre de préfixes externes³) :

$$P_{par_paquet} = 0.16 + 155 \times 10^{-6} \times 10^6 = \boxed{155.16 \text{ W}}$$

Pour $alpm$:

$$P_{par_paquet} = 0.54 + 48 \times 10^{-6} \times 10^6 = \boxed{48.16 \text{ W}}$$

Nous allons donc utiliser $alpm$ pour maximiser les chances du code sans cache.

Pour un cache de taille 10^6 :

$$P_{par_paquet} = 0.21 + 5 \times 10^{-6} \times 10^6 = \boxed{5.21 \text{ W}}$$

3. <https://www.cidr-report.org/as2.0/> : plus de 1 000 000 de préfixes sont recensés.

et donc pour une table *alpm* et le cache : $48.16 + 5.21 = 53.37$ W.

Le traitement d'un paquet pour le programme sans cache va donc être

$$234.769 \times 10^{-9} \text{ secondes} \times 48.16 \text{ W} = \boxed{113 \times 10^{-7} \text{ Joules}}$$

Celui d'un paquet pour le programme avec le cache si l'entrée n'est pas en cache :

$$(1.182 \times 234.769 \times 10^{-9} \text{ secondes}) \times 53.37 \text{ W} = \boxed{148 \times 10^{-7} \text{ Joules}}$$

Celui d'un paquet pour le programme avec le cache si l'entrée est en cache :

$$234.769 \times 10^{-9} \text{ secondes} \times 5.21 \text{ W} = \boxed{12 \times 10^{-7} \text{ Joules}}$$

On retire le paramètre 1.182 (qui correspond à une estimation de l'allongement du traitement dans le temps dû aux opérations qui alimentent le cache) dans la dernière expression car la table *alpm* n'est pas accédée, la mémoire n'est pas alimentée et le paquet n'est pas ralenti.

Avec ces chiffres, si l'on estime la consommation due à un flux de 1Go au total (en-têtes compris), avec une chance de $\frac{1}{100000}$ d'ajouter une entrée au cache, on obtient, en moyenne, le résultat de la Figure 8 :

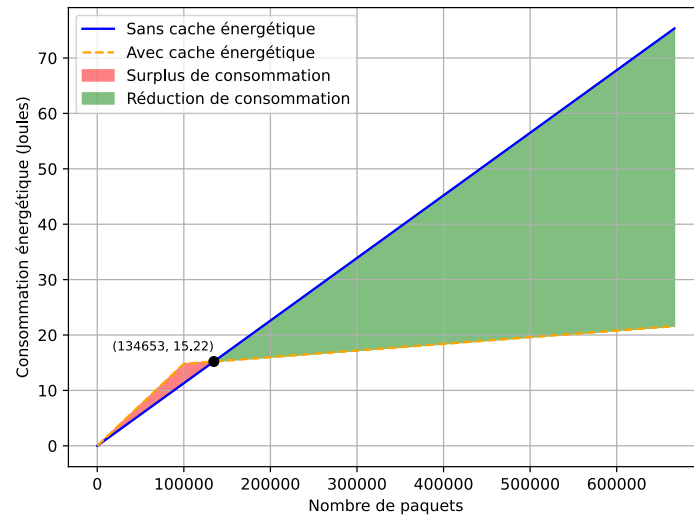


FIGURE 9 – Consommation énergétique de l'envoi d'1 Go par paquets de 1500 octets. Dans la version avec le cache énergétique : l'entrée est ajoutée dans le cache au 100000^{ième} paquet.

Sur la Figure 8 on voit que la neutralité est atteinte, en moyenne, au paquet numéro 134653, soit environ après avoir envoyé 201Mo. On voit aussi que la version avec cache a consommé 22 Joules contre 75 Joules pour la version sans cache, soit une réduction de 70%.

Il peut être intéressant de connaître la probabilité de consommer plus que ce que l'on économise. Il suffit alors de résoudre (pour 1Go de paquets de 1500 octets, 666666 paquets au total) :

$$666666 \times 113 \times 10^{-7} = (148 \times 10^{-7} \times N) + (12 \times 10^{-7} \times (666666 - N))$$

$$\iff \boxed{N = 495098}$$

où N est le nombre de paquets qui font partie du même flux mais qui n'ont pas été ajoutés au cache.

Ici nous sommes dans une distribution géométrique, équivalente à la question "combien de dés dois-je lancer avant d'avoir une certaine face ?". Le dé a 100000 faces et la question est "quelle est la probabilité d'obtenir la bonne face du dé au bout de 495098 lancers ou plus ?". La réponse est :

$$\mathbb{P}(X \geq k) = (1 - p)^k$$

$$\iff \mathbb{P}(X \geq 495098) = \left(1 - \frac{1}{100000}\right)^{495098} = 0.007 = \boxed{0.7\%}$$

Finalement, les principaux résultats de cette solution sont :

- En moyenne on réduit de 70% la consommation énergétique ;
- Étant donné que la solution avec cache consomme plus pour les paquets dont l'adresse de destination n'est pas dans le cache : en moyenne, la neutralité entre la solution avec et sans cache apparaît après avoir traité 201Mo du même flux ;
- La probabilité que la consommation soit plus élevée avec le cache que sans est de 0.7%.

6.3.4 Précisions concernant les limites et perspectives des implémentations

En cas de modifications des tables de routage dans le plan de contrôle, et donc dans la table *lpm* utilisée pour peupler le cache énergétique, il faut également mettre à jour le cache énergétique. Le cache peut être remis à 0 pour s'assurer qu'aucun comportement non-voulu n'apparaisse, typiquement une incohérence entre la table de routage et le cache énergétique. Une autre possibilité serait de cibler les entrées qui ont été modifiées dans la table de routage. Par exemple, supposons que la table *lpm* contienne deux entrées : 10.0.0.0/8 -> port_8 et 10.0.0.0/24 -> port_24, et que l'entrée suivante est présente en cache : 10.0.0.42 -> port_24. Si l'on retire l'entrée 10.0.0.0/24 -> port_24, faut-il retirer l'entrée en cache ? la modifier ?

Ces solutions ne sont pas mutuellement exclusives, on peut très bien imaginer une solution qui emploie plusieurs solutions à la fois. Par exemple, réutiliser le résultat du cache énergétique pour savoir quelles entrées sont le plus populaire pour la solution de découpage et ré-ordonnancement des tables.

L'archive⁴ contient une preuve de concept d'un cache énergétique probabiliste dans laquelle j'ai ajouté une table à correspondance exacte, un système de recrutement aléatoire et l'ajout de nouvelles entrées dans le cache à l'aide du recrutement. L'archive⁵ contient un outil optimisant un programme P4 rédigé par Jean-Romain LUTTRINGER. Il transforme le code P4 en une version équivalente consommant moins d'énergie. Par exemple, dans sa version actuelle, il transforme les tables *lpm* en tables *alpm*. Le but étant d'inclure toutes les optimisations citées plus haut dans cet outil, une fois qu'elles sont validées et ajustées.

4. <https://filesender.renater.fr/?s=download&token=a02ca982-677d-4be6-8e4b-d1c2acf75c4b> *FileSender* est un service hébergé par *RENATER*.

5. <https://filesender.renater.fr/?s=download&token=65689543-af7f-4fc7-a642-730c8a3721dc> *FileSender* est un service hébergé par *RENATER*.

7 Conclusion

Durant ce stage, j'ai beaucoup appris concernant les réseaux programmables. Que ce soit leur fonctionnement à travers la prise en main de Tofinos, leur consommation énergétique à travers la lecture d'articles ou leur optimisation à travers la création de nouvelles méthodes de réduction de la consommation. Cet apprentissage me sera très utile pour la thèse que je ferai sur la même thématique.

J'ai eu la chance d'améliorer mes compétences nécessaires à un travail de recherche notamment la communication, via des présentations fréquentes avec mes encadrants ou avec l'équipe de recherche, mais aussi la rigueur scientifique et le développement d'un esprit critique. En concrétisant des idées abstraites comme la création de solutions de choix d'entrées à mettre en cache, j'ai travaillé ma créativité et mes compétences techniques en réseaux programmables (P4, routage, optimisations de code), en études de la performance et en statistiques (*Random Forest*, *Permutation Feature Importance*).

J'ai préparé un modèle d'estimation de la consommation énergétique d'équipements P4-programmables et j'ai également préparé des optimisations. Le modèle repose sur le constat que la consommation énergétique varie en fonction du programme déployé sur les commutateurs programmables. Parmi la centaine de paramètres récoltés pour modéliser la consommation, une étude statistique a révélé que l'utilisation de la mémoire TCAM était la principale responsable de la consommation en énergie.

Les optimisations proposées essayent donc de réduire l'utilisation de cette mémoire, en dépliant les préfixes, découpant et réordonnant les tables et surtout en introduisant un cache énergétique qui stocke le résultat de précédentes recherches en TCAM dans une mémoire SRAM plus économe. Je n'ai malheureusement pas pu tester et finaliser mon travail car je n'avais pas accès aux données relatives à la consommation énergétique des équipements que je manipulais.

J'ai tout de même eu la chance de manipuler des équipements réseau programmables, c'est une expérience qui sera forcément bénéfique dans le futur, que ce soit pour ma thèse ou pour d'autres travaux qui bénéficieraient d'une expérience sur du matériel.

7.1 Perspectives

Les optimisations et le modèle présenté précédemment nécessitent encore des résultats expérimentaux pour pouvoir analyser leurs qualités et défauts. Obtenir des résultats pratiques est l'une des premières choses que je ferai lorsque je continuerai à travailler sur ce projet.

7.1.1 Perspectives à court terme

Afin d'obtenir des résultats, j'utiliserai la plateforme *SlicesFR*, qui compte plusieurs Tofinos. Chaque Tofino peut être réservé pour pouvoir expérimenter sur un réseau programmable.

Pour générer différents débits, je compte utiliser une méthode d'amplification de trafic basée sur la programmabilité des Tofinos [Lima], schématisée dans la Figure 9. En définissant un groupe multicast, le Tofino duplique des paquets sur plusieurs chaîne de traitement. Chacune de ces chaîne de traitement redirige alors le trafic sur le port connecté à la destination que l'on souhaite profiler.

L'utilisation de différents types de trafic à l'aide d'outils appropriés (*iperf*¹, *tcpreplay*²) permettra de

1. <https://iperf.fr/> : est un outil de génération de trafic, particulièrement intéressant pour générer un flux stable de paquets UDP.

2. <https://tcpreplay.appneta.com/> : est un outil de jeu de traces.

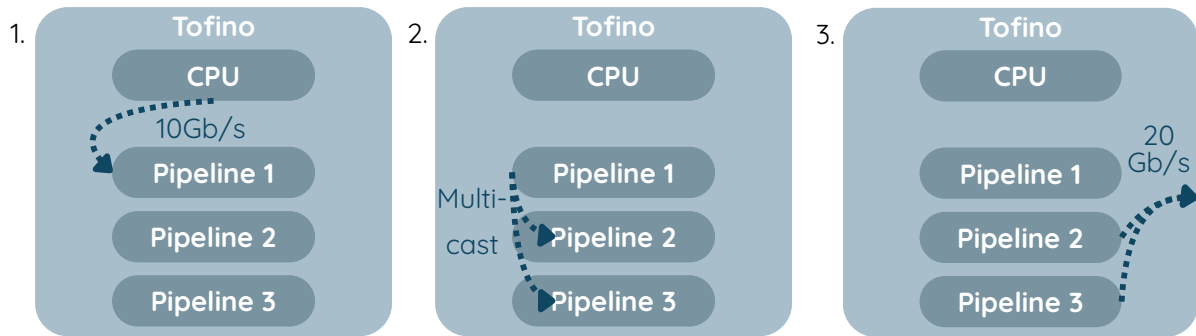


FIGURE 10 – Schéma du fonctionnement de l'amplification de trafic à l'aide des Tofinos.

valider les estimations de consommation énergétique et de vérifier l'impact du type de trafic.

Une fois que les résultats expérimentaux seront présents dans le jeu de données, je vérifierai si un modèle *random forest* est approprié pour estimer la consommation d'équipements P4-programmables. Il sera intéressant de comparer les nouveaux résultats avec ceux présentés dans [Lim] afin d'avoir une idée de la précision du modèle. La *Permutation Feature Importance* servira pour savoir quels leviers sont les plus importants à optimiser.

Concernant les différentes optimisations proposées, le WIDE Project³ met à disposition des traces anonymisées de trafic observé sur des réseaux d'opérateurs. Elles représentent un échantillon des communications qui transitent sur internet, on devrait donc y retrouver les propriétés concernant le caractère volumineux des flux. Je comparerai mes propositions de cache avec celles visant à réduire la consommation énergétique [LVCG05] mais aussi celles qui cherchent à réduire la latence [XCH⁺09], [ZBZ⁺18] et [XQH⁺23].

7.1.2 Perspectives à long terme

Ce stage est une préparation à un doctorat qui portera sur le même sujet : l'économie d'énergie dans les réseaux informatiques. Pendant mon doctorat, j'étudierai les protocoles visant à réduire la consommation énergétique des réseaux, ainsi que l'auto-régulation d'un réseau en fonction de directives. Un exemple de famille de protocoles est l'extinction de liens et l'extinction de nœuds. Le but étant d'éteindre un maximum de liens, ou nœuds, tout en ayant une qualité de service suffisante. Par exemple, [RJV24] proposent un algorithme d'auto-régulation de réseaux qui éteint des liens en fonction du trafic à acheminer.

La dégradation de qualité

Les réseaux programmables permettent la création d'une nouvelle famille de protocoles : la dégradation de qualité. En réduisant le nombre de services non-nécessaires, on s'attend à observer une réduction de la consommation. Un exemple de service non-nécessaire est l'équilibrage de charge, il vise à réduire la congestion mais n'est pas nécessaire au routage. En dégradant la qualité du commutateur, c'est-à-dire en lui retirant ses capacités d'équilibrage de charge, on devrait observer une réduction de la consommation énergétique. Le principal défi ici est de ne pas détériorer l'expérience utilisateur. Le but long terme serait de créer une solution qui choisisse automatiquement et de manière optimale les services à garder et ceux qu'il est possible d'éteindre en fonction de directives.

Auto-régulation selon des directives

La programmabilité des réseaux permet également une régulation précise et globale. Elle est précise car il devient possible d'adapter des éléments particuliers comme la taille des tables au nombre d'entrées

3. <http://mawi.wide.ad.jp/mawi/> Le WIDE Project est un consortium global qui promeut la recherche relative à internet, ses technologies, ses standards et la communication entre la recherche académique et industrielle.

près. Elle est globale car tous les services et protocoles peuvent être modifiés ou supprimés. Cette autorégulation peut prendre la forme d'un cycle circadien⁴ pour les commutateurs. Elle éteindrait des liens et nœuds la nuit, elle réduirait également la taille des caches énergétiques et la solution de dégradation supprimerait la majorité des services.

Les directives peuvent suivre les résultats d'une analyse longue durée, comme par exemple le cycle jour/nuit. Elles peuvent également suivre une analyse temps réel à l'aide d'outils de supervision donnant des informations concernant le nombre de flux, leur volume, la congestion subie *etc.* Il serait également intéressant d'étudier des outils de prédiction de trafic afin de pouvoir prédire les congestions plutôt que de les subir.

À noter que ces objectifs peuvent être étudiés dans des contextes différents, *e.g.*, les routeurs de bordure possèdent énormément de préfixes dans leur table de routage et sont donc de parfaits candidats pour le cache énergétique, ce qui n'est pas le cas d'autres équipements. L'outil d'extinction de commutateurs P4Green [GK23] ne fonctionne que dans les centres de données. Les différentes topologies et caractéristiques des équipements sont à prendre en compte.

Il est également intéressant de noter que les problématiques soulevées révèlent des défis d'algorithmique distribuée, ainsi que des défis d'optimisation. Sans consensus il est difficile de choisir les liens, ou nœuds, à éteindre, sinon le réseau risque d'être scindé en plusieurs sous-réseaux. Le choix des services à retirer ou à ajouter pour correspondre à des directives est un problème d'optimisation qu'il faudra sûrement approximer à l'aide d'heuristiques.

4. Un cycle circadien correspond au cycle jour/nuit des êtres vivants. Le corps humain est plus actif le jour que la nuit par exemple.

Bibliographie

- [AMD] AMD. AMD Vivado™ Design Suite. <https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vivado.html>.
- [BDB⁺11] Raffaele Bolla, Franco Davoli, Roberto Bruschi, Ken Christensen, Flavio Cucchietti, and Suresh Singh. The potential impact of green technologies in next-generation wireline networks : Is there room for energy saving optimization? *IEEE Communications Magazine*, 49(8) :80–86, 2011.
- [BDG⁺14] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. P4 : programming protocol-independent packet processors. *SIGCOMM Comput. Commun. Rev.*, 44(3) :87–95, July 2014.
- [BGK⁺13] Pat Bosshart, Glen Gibb, Hun-Seok Kim, George Varghese, Nick McKeown, Martin Izzard, Fernando Mujica, and Mark Horowitz. Forwarding metamorphosis : Fast programmable match-action processing in hardware for sdn. In *ACM SIGCOMM 2013*. ACM, August 2013.
- [Cal90] R. Callon. Use of OSI IS-IS for routing in TCP/IP and dual environments. RFC 1195, December 1990.
- [Cis] Cisco. Select BGP Best Path Algorithm. https://www.cisco.com/c/en/us/support/docs/ip/border-gateway-protocol-bgp/13753-25.html?utm_source=chatgpt.com.
- [CLX⁺23] Xiang Chen, Hongyan Liu, Qingjiang Xiao, Jianshan Zhang, Qun Huang, Dong Zhang, Xuan Liu, and Chunming Wu. Melody : Toward resource-efficient packet header vector encoding on programmable switches. In *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*, page 1–10, May 2023.
- [eF] eBPF Foundation. Dynamically program the kernel for efficient networking, observability, tracing, and security. <https://ebpf.io/>.
- [ene10] Ieee standard for information technology– local and metropolitan area networks– specific requirements– part 3 : Cdma/cd access method and physical layer specifications amendment 5 : Media access control parameters, physical layers, and management parameters for energy-efficient ethernet. *IEEE Std 802.3az-2010 (Amendment to IEEE Std 802.3-2008)*, pages 1–302, 2010.
- [FOZ⁺24] David Franco, Eder Ollora Zaballa, Mingyuan Zang, Asier Atutxa, Jorge Sasiain, Aleksander Pruski, Elisa Rojas, Marivi Higuero, and Eduardo Jacob. A comprehensive latency profiling study of the tofino p4 programmable asic-based hardware. *Computer Communications*, 218 :14–30, 2024.
- [FOZZ⁺24] David Franco, Eder Ollora Zaballa, Mingyuan Zang, Asier Atutxa, Jorge Sasiain, Aleksander Pruski, Elisa Rojas, Marivi Higuero, and Eduardo Jacob. A comprehensive latency profiling study of the tofino p4 programmable asic-based hardware. *Computer Communications*, 218 :14–30, 2024.
- [FRD19] Aaron Fisher, Cynthia Rudin, and Francesca Dominici. All models are wrong, but many are useful : Learning a variable’s importance by studying an entire class of prediction models simultaneously, 2019.
- [FSDC90] Mark Fedor, Martin Lee Schoffstall, James R. Davin, and Dr. Jeff D. Case. Simple Network Management Protocol (SNMP). RFC 1157, May 1990.
- [Gel23] Erol Gelenbe. Electricity consumption by ict : Facts, trends, and measurements. *Ubiquity*, 2023(August), August 2023.

- [GK23] Garegin Grigoryan and Minseok Kwon. Towards greener data centers via programmable data plane. In *2023 IEEE 24th International Conference on High Performance Switching and Routing (HPSR)*, page 62–67, Albuquerque, NM, USA, June 2023. IEEE.
- [GMMO10] Frederic Giroire, Dorian Mazauric, Joanna Moulrierac, and Brice Onfroy. Minimizing routing energy consumption : From theoretical to practical results. page 252–259, Hangzhou, China, 2010. IEEE.
- [HHM⁺21] Frederik Hauser, Marco Häberle, Daniel Merling, Steffen Lindner, Vladimir Gurevich, Florian Zeiger, Reinhard Frank, and Michael Menth. A survey on data plane programming with P4 : fundamentals, advances, and applied research. *CoRR*, abs/2101.10632, 2021.
- [HHM⁺23] Frederik Hauser, Marco Häberle, Daniel Merling, Steffen Lindner, Vladimir Gurevich, Florian Zeiger, Reinhard Frank, and Michael Menth. A survey on data plane programming with p4 : Fundamentals, advances, and applied research. *Journal of Network and Computer Applications*, 212 :103561, 2023.
- [IEA] IEA. Data Centres and Data Transmission Networks. <https://www.iea.org/energy-system/buildings/data-centres-and-data-transmission-networks>.
- [Inta] Intel. Intel® P4 Suite for FPGA. <https://www.intel.com/content/www/us/en/software/programmable/p4-suite-fpga/overview.html>.
- [Intb] Intel. Intel® Tofino™ . <https://www.intel.com/content/www/us/en/products/details/network-io/intelligent-fabric-processors/tofino.html>.
- [JLV23] Romain Jacob, Jackie Lim, and Laurent Vanbever. Does rate adaptation at daily timescales make sense? In *Proceedings of the 2nd Workshop on Sustainable Computer Systems*, page 1–7, Boston MA USA, July 2023. ACM.
- [JS94] Theodore Johnson and Dennis Shasha. 2q : A low overhead high performance buffer management replacement algorithm. In *Very Large Data Bases Conference*, 1994.
- [Jun] Juniper. BGP Overview. https://www.juniper.net/documentation/us/en/software/junos/bgp/topics/topic-map/bgp-overview.html?utm_source=chatgpt.com.
- [JV23] Romain Jacob and Laurent Vanbever. The internet of tomorrow must sleep more and grow old. *ACM SIGEnergy Energy Informatics Review*, 3(3) :27–32, 2023.
- [LGZ⁺22] Yifan Li, Jiaqi Gao, Ennan Zhai, Mengqi Liu, Kun Liu, and Hongqiang Harry Liu. Cetus : Releasing p4 programmers from the chore of trial and error compiling. page 371–385, 2022.
- [Lima] Jackie Lim. How much does it burn ? profiling the energy model of a tofino switch. page 33 p. Artwork Size : 33 p. Medium : application/pdf Publisher : ETH Zurich.
- [Limb] Jackie Lim. Power modelling framework for network switches.
- [LVCG05] Bengu Li, Ganesh Venkatesh, Brad Calder, and Rajiv Gupta. Exploiting a computation reuse cache to reduce energy in network processors. In Tom Conte, Nacho Navarro, Wen-mei W. Hwu, Mateo Valero, and Theo Ungerer, editors, *High Performance Embedded Architectures and Compilers*, page 251–265, Berlin, Heidelberg, 2005. Springer.
- [MAB⁺08] Nick McKeown, Tom Anderson, Hari Balakrishnan, Guru Parulkar, Larry Peterson, Jennifer Rexford, Scott Shenker, and Jonathan Turner. Openflow : enabling innovation in campus networks. *SIGCOMM Comput. Commun. Rev.*, 38(2) :69–74, March 2008.
- [MHAT14] Arun Vishwanath Member, Kerry Hinton, Robert W. A. Ayre, and Rodney S. Tucker. Modeling energy consumption in high-capacity routers and switches. *IEEE Journal on Selected Areas in Communications*, 32(8) :1524–1532, 2014.
- [Moy98] John Moy. OSPF Version 2. RFC 2328, April 1998.
- [OOW93] Elizabeth J. O’Neil, Patrick E. O’Neil, and Gerhard Weikum. The lru-k page replacement algorithm for database disk buffering. *SIGMOD Rec.*, 22(2) :297–306, June 1993.
- [RJV24] Lukas Röllin, Romain Jacob, and Laurent Vanbever. A sleep study for isp networks : Evaluating link sleeping on real world data. page 8 p., 2024. Artwork Size : 8 p. Medium : application/pdf Publisher : ETH Zurich.
- [RMHL12] P. Reviriego, J.A. Maestro, J.A. Hernández, and D. Larrabeiti. Study of the potential energy savings in ethernet by combining energy efficient ethernet and adaptive link rate. *Transactions on Emerging Telecommunications Technologies*, 23(3) :227–233, 2012.
- [San] Sandvine. https://www.applogicnetworks.com/hubfs/Sandvine_Redesign_2019/Downloads/2024/GIPR/GIPR%202024.pdf.

- [SPL⁺16] Takayuki Sasaki, Christos Pappas, Taeho Lee, Torsten Hoefler, and Adrian Perrig. Sdnsec : Forwarding accountability for the SDN data plane. *CoRR*, abs/1605.01944, 2016.
- [Sta] Statista. Leading countries by number of data centers as of March 2025. <https://www.statista.com/statistics/1228433/data-centers-worldwide-by-country/>.
- [SVZR11] Vijay Sivaraman, Arun Vishwanath, Zhi Zhao, and Craig Russell. Profiling per-packet and per-byte energy consumption in the netfpga gigabit router. In *2011 IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, page 331–336, April 2011.
- [WADV20] Patrick Wintermeyer, Maria Apostolaki, Alexander Dietmüller, and Laurent Vanbever. P2go : P4 profile-guided optimizations. In *Proceedings of the 19th ACM Workshop on Hot Topics in Networks*, page 146–152, Virtual Event USA, November 2020. ACM.
- [Wan] Henry Wang. Algorithmic longest prefix matching in programmable switch.
- [Wis] WisdomAnswer. How many datacenters are there in 2020 ? . <https://wisdomanswer.com/how-many-data-centers-are-there-in-2020/>.
- [WLJ⁺24] Chenan Wen, Zhuocong Li, Syed Usman Jafri, Xiaokang Qiu, and Sanjay Rao. P4cgo : Control plane guided p4 program optimization. In *Proceedings of the SIGCOMM Workshop on Formal Methods Aided Network Operation*, page 1–7, Sydney NSW Australia, August 2024. ACM.
- [XCH⁺09] Bo Xu, Jian Chang, Shimeng Huang, Yibo Xue, and Jun Li. Efficiency of cache mechanism for network processors. *Tsinghua Science and Technology*, 14(5) :575–585, October 2009.
- [XQH⁺23] Jiarong Xing, Yiming Qiu, Kuo-Feng Hsu, Songyuan Sui, Khalid Manaa, Omer Shabtai, Yonatan Piasetzky, Matty Kadosh, Arvind Krishnamurthy, T. S. Eugene Ng, and Ang Chen. Unleashing smartnic packet processing performance in p4. In *Proceedings of the ACM SIGCOMM 2023 Conference*, page 1028–1042, New York NY USA, September 2023. ACM.
- [ZBZ⁺18] Cheng Zhang, Jun Bi, Yu Zhou, Keyao Zhang, and Zijun Ma. B-cache : A behavior-level caching framework for the programmable data plane. In *2018 IEEE Symposium on Computers and Communications (ISCC)*, page 00084–00090, 2018.
- [ZCQ⁺23] Zijie Zeng, Lin Cui, Mimi Qian, Zhen Zhang, and Kaimin Wei. A survey on sliding window sketch for network measurement. *Computer Networks*, 226 :109696, 2023.